

# Recursive Sorting Techniques

20 March 2026 20:31

## Quick Sort:

Quick Sort is a Divide and Conquer Sorting Algorithm

1. Pick a pivot element
2. Partition the array so:
  - i. The elements smaller than pivot are on the left
  - ii. Element greater than pivot are on the right
3. Recursively apply the same process on left and right parts.

|   |   |   |   |   |    |   |
|---|---|---|---|---|----|---|
| 0 | 1 | 2 | 3 | 4 | 5  | 6 |
| 8 | 3 | 1 | 7 | 0 | 10 | 2 |

Choose the pivot element (last element  $\rightarrow 2$ )

|   |   |   |   |   |    |   |
|---|---|---|---|---|----|---|
| 0 | 1 | 2 | 3 | 4 | 5  | 6 |
| 2 | 3 | 1 | 7 | 0 | 10 | 8 |

pivot = 2  
Li = 0  
Hi =

```
package DivideAndConquer;
```

```
import java.util.*;
```

```
public class QuickSort {
```

```
    public void quickSort(int A[], int low, int high){
```

```
        if(low < high){
```

```
            int pi = partition(A, low, high);
```

```
            //recursively sort elements
```

```
            quickSort(A, low, pi - 1);
```

```
            quickSort(A, pi + 1, high);
```

```
        }
```

```
    }  
    public int partition(int[] A, int low, int high){
```

```
        int pivot = A[high]; //choose the last element
```

```
        int i = low - 1; //index of smaller element
```

```
        for(int j = low; j < high; j++){
```

```
            if(A[j] < pivot){
```

```
                i++;
```

```
                //swap A[i] and A[j]
```

```
                int t = A[i];
```

```
                A[i] = A[j];
```

```
                A[j] = t;
```

```
            }
```

```
        }  
        //place the pivot at correct position
```

```
        int t = A[i + 1];
```

```
        A[i + 1] = A[high];
```

```
        A[high] = t;
```

```
        return i + 1;
```

```
    }
```

```
    public void printArray(int[] A, String msg){
```

```
        System.out.println(msg + Arrays.toString(A));
```

```
    }
```

```
    public static void main(String[] args){
```

```
        Random r = new Random();
```

0

0

1

2

3

4

5

6

|   |   |   |   |   |    |   |
|---|---|---|---|---|----|---|
| 8 | 3 | 1 | 7 | 0 | 10 | 2 |
|---|---|---|---|---|----|---|

low = 0  
high = 6  
pi = 2

low = 0  
high = 1  
pi = 1

low = 0  
high = 0  
pi = 0

quickSort(A, 3, 6)

quickSort(A, 2, 6)

|   |   |   |   |   |   |    |
|---|---|---|---|---|---|----|
| 0 | 1 | 2 | 3 | 4 | 5 | 6  |
| 0 | 1 | 2 | 7 | 3 | 8 | 10 |

low = 0

high = 6

pivot = 2

i = -1

j = 0 1 2 3 4 5 6

t = 8

low = 0

high = 1

pivot = 0

i = -1

j = 0 1

t = 1

```
int A[] = new int[10];
int i;
for(i = 0; i < A.length; i++){
    A[i] = r.nextInt(10, 100);
}
QuickSort q = new QuickSort();
q.printArray(A, "Array before sorting: ");
q.quickSort(A, 0, A.length-1);
q.printArray(A, "Array after sorting.");
}
```